

Customize Your Storefront

Modify the storefront by adding a custom page and component.

Prerequisites

- Generate a Storefront Next app by following the steps in [Create a Storefront in Business Manager with Local Code](#) or [Explore Storefront Next Code Locally](#).
- An Integrated Development Environment (IDE), such as VS Code, Cursor, or your favorite IDE. See [Download VS Code](#) or [Download Cursor](#).

Steps

1. Create a component that displays a banner. Later, add this component to a new page.
 - a. In `{project-folder}/src/components`, create a folder with the name `custom-banner`.
 - b. In the new `custom-banner` folder, create the `index.tsx` file with this code.

```
1 import { type ReactElement } from 'react';
2
3 export interface CustomBannerProps {
4   /** The title text to display in the banner
5    title: string;
6   /** Optional subtitle text */
7   subtitle?: string;
8   /** Optional CSS class name for custom styli
9   className?: string;
10 }
11
12 /**
13  * CustomBanner component that displays a promine
14  *
15  * @param props - The component props
16  * @param props.title - The title text to display
17  * @param props.subtitle - Optional subtitle text
18  * @param props.className - Optional CSS class na
19  *
20  * @returns A React element containing the banner
21  *
22  * @example
23  * ```tsx
24  * <CustomBanner title="Welcome to the Custom Col
25  * ```
26  *
27  * @example
28  * ```tsx
```

Contents

[Prerequisites](#)[Steps](#)[Next Steps](#)



```
34     <div className={wFull bgPrimary text-  
35         <div className="container mx-auto px  
36             <div className="max-w-4xl mx-aut  
37                 <h1 className="text-4xl sm:t  
38                     {title}  
39                 </h1>  
40                     {subtitle && (  
41                         <p className="text-lg sm  
42                             {subtitle}  
43                         </p>  
44                     )}  
45                 </div>  
46             </div>  
47         </div>  
48     );  
49 }
```

The `CustomBanner` component in `/custom-banner/index.tsx` renders a centered banner with a large title and optional subtitle. It accepts `title` (required), `subtitle` (optional), and `className` (optional) via `CustomBannerProps`. It uses Tailwind classes for responsive typography, primary theme colors, and spacing, and centers content within a max-width container. The component returns a React element with the title as an `<h1>` and conditionally renders the subtitle as a paragraph, if provided.

2. Add a page that includes this new component. The new page is created as a route file. In `{project-folder}/src/routes`, create a route file with the name `_app.custom-page.tsx` and add this code.

```
1 import { type ReactElement } from 'react';  
2 import { createPage, type RouteComponentProps }  
3 import CustomBanner from '@components/custom-ba  
4  
5 /**  
6  * Data structure for the custom-page.  
7  * This page doesn't require any loader data, so  
8  */  
9 export type CustomPageData = Record<string, neve  
10  
11 /**  
12  * Custom page view component that displays a ban  
13  * @returns JSX element representing the custom p  
14  */  
15 // eslint-disable-next-line react-refresh/only-e  
16 function CustomPageView({}: RouteComponentProps<  
17     return (  
18         <div className="pb-16">  
19             <CustomBanner title="Welcome to the  
20         </div>  
21     );  
22 }  
23  
24 /**  
25  * Custom page component with loading fallback.  
26  * This creates a page component that wraps the C  
27  * The createPage utility provides consistent loa  
28  * @returns Page component with custom page funct  
29  */  
30 // eslint-disable-next-line react-refresh/only-e
```



page at `/custom-page`. The `_app.` prefix in the file name causes the React Router to nest your custom page route under the `_app.tsx` layout. That layout renders the header, your page content, and the footer. It exports a default page via `createPage`, which wraps `CustomPageView` with `Suspense` and loading handling. `CustomPageView` renders the `CustomBanner` component inside a container with bottom padding. The page uses `CustomPageData`, an empty record. It doesn't require loading data so it renders immediately without async data fetching.

3. In a terminal window, go to the storefront folder.

```
1 cd {project-folder}
```

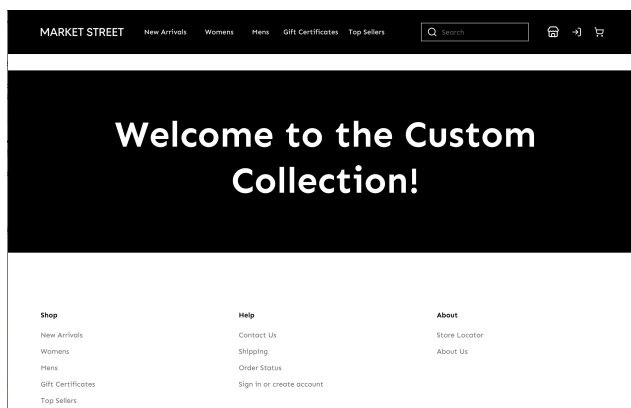
4. If you haven't already, install Storefront Next project dependencies that are specified in `package.json`.

```
1 pnpm install
```

5. Build the storefront and then run the development server.

```
1 pnpm run build
2 pnpm run dev
```

6. Navigate to the new page that you created by appending `/custom-page` to the storefront URL. For example, go to this default URL: `http://localhost:5173/custom-page`. The new page shows the banner with a title.



Next Steps

- [Deploy Storefront Next to Managed Runtime](#)



Try for free



DEVELOPER CENTERS

Heroku
MuleSoft
Tableau
Commerce Cloud
Lightning Design System
Einstein
Quip

POPULAR RESOURCES

Documentation
Component Library
APIs
Trailhead
Sample Apps
AppExchange

COMMUNITY

Trailblazer Commu
Events and Calend
Partner Community
Blog
Salesforce Admins
Salesforce Architec

© Copyright 2026 Salesforce, Inc. [All rights reserved](#). Various trademarks held by their respective owners.
Salesforce, Inc. Salesforce Tower, 415 Mission Street, 3rd Floor, San Francisco, CA 94105, United States

[Legal](#) [Terms of Service](#) [Privacy Information](#) [Responsible Disclosure](#) [Trust](#) [Contact](#)

[Use of Cookies](#) [Cookie Preferences](#)  [Your Privacy Choices](#)